

Convolutions

Name: _____

NetID: _____

Section: _____

1. CNN shapes, parameters, and receptive fields

Input shape: $(B, 1, 61, 125)$. Here B is the number of images in a batch; the remaining axes are channels, height, and width (1 grayscale channel, 61 rows, 125 columns). Every convolution uses a 3×3 kernel, no padding, and a bias, followed by ReLU. Each output channel uses all input channels. ReLU adds no learned parameters.

Calculation aids.

$$y_h = \left\lfloor \frac{x_h + 2p_h - f_h + s_h}{s_h} \right\rfloor.$$

x_h : input height; y_h : output height; f_h : kernel height; p_h : zero-padding rows on *each* side; s_h : vertical stride. $\lfloor \cdot \rfloor$: round down to an integer. Replace subscript h (height) by w (width) to compute y_w from x_w, f_w, p_w, s_w .

$$n_{\text{weights}} = f_h f_w C D, \quad n_{\text{biases}} = D.$$

$n_{\text{weights}}, n_{\text{biases}}$: numbers of learned weights and biases; f_h, f_w : kernel height and width.

C : input channels; D : output channels. There is one bias per output channel.

Receptive field. Start with one output unit and trace backward to the input:

$$q_{\text{prev}} = (q - 1)s + f.$$

q : span of the selected positions in the current map (start at $q = 1$); q_{prev} : their covered span in the preceding map; s : stride of the layer being reversed; f : its kernel size along the same axis. Repeat using the new span as q . For our square kernels, $f = f_h = f_w$; equal strides use $s = s_h = s_w$. These spans count positions along one axis, not the total area.

A. Fill the empty cells. Conv1 is supplied; RF side is the input-region width (equal to its height).

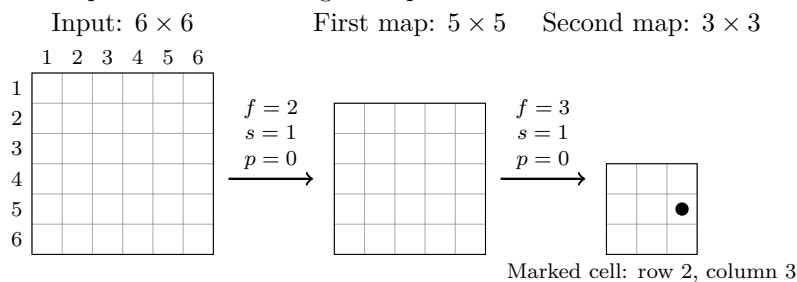
Layer	$C \rightarrow D$	s	Output $y_h \times y_w$	Weights	Biases	RF side
Conv1	$1 \rightarrow 16$	1	59×123	144	16	3
Conv2	$16 \rightarrow 16$	2				
Conv3	$16 \rightarrow 24$	1	27×59	3,456	24	

Show the calculation for **Conv2 output height** and **Conv3 receptive field**:

B. Quick checks. If B doubles, do convolution parameter counts double? _____

The RF in the Conv3 row describes: ☐ one Conv3 unit ☐ the final scalar prediction.

C. Draw the receptive field. Each layer below has one channel. For the marked cell, outline its input cells in the first map, then shade its receptive field in the original input. Rows count down and columns right, starting at 1.



From feature maps back to pixels

2. Upconvolution and VisualBackProp

Method reminder. For one image, reuse the post-ReLU maps from the prediction's forward pass. First average each layer's channels:

$$\bar{A}_\ell(p) = \frac{1}{C_\ell} \sum_{c=1}^{C_\ell} a_\ell(c, p).$$

ℓ : convolution-layer index; c : channel index; p : spatial position (not padding); $a_\ell(c, p)$: post-ReLU activation; C_ℓ : number of output channels at layer ℓ (Part 1's D); $\bar{A}_\ell(p)$: channel average. $\sum$: add over all channels.

Start at the deepest layer, then upscale and multiply toward the input:

$$M_L = \bar{A}_L, \quad M_{\ell-1} = \bar{A}_{\ell-1} \odot \text{Deconv}_{K_\ell, S_\ell}(M_\ell), \quad \ell = L, \dots, 2.$$

L : deepest layer ($L = 3$ here); M_ℓ : intermediate mask at layer ℓ ; $\bar{A}_{\ell-1}$: preceding layer's average map; $\odot$: multiply matching entries. Deconv: transposed convolution with all kernel weights 1 and bias 0; K_ℓ : square kernel side length; S_ℓ : stride of forward layer ℓ . Thus K_ℓ and S_ℓ correspond to Part 1's f and s , not learned weights.

Finally, upscale M_1 with Deconv_{K_1, S_1} to input size and normalize to $[0, 1]$. There is no multiplication by the original image.

For a reverse step with no padding, compute its height using

$$y_h = (x_h - 1)S_\ell + K_\ell.$$

x_h : current mask height; y_h : upsampled height; K_ℓ : kernel side length; S_ℓ : stride. Replace h by w for widths (x_w, y_w). Use the same kernel size and stride as forward layer ℓ .

A. Hand-compute overlapping contributions. For this *toy example only*, use $K_\ell = 2$, $S_\ell = 1$, no padding, and zero bias:

$$M_\ell = \begin{bmatrix} 1 & 2 \end{bmatrix}, \quad U_{\ell-1} = \text{Deconv}_{K_\ell, S_\ell}(M_\ell).$$

M_ℓ : current mask; $U_{\ell-1}$: temporary upsampled map before multiplication; $K_\ell = 2$: a 2×2 all-ones kernel; $S_\ell = 1$: stride 1. Each mask value places a scaled copy of the kernel; overlapping entries add.

Fill the upsampled map and the next mask:

$$U_{\ell-1} = \begin{bmatrix} \text{---} & \text{---} & \text{---} \\ \text{---} & \text{---} & \text{---} \end{bmatrix}, \quad \bar{A}_{\ell-1} = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix}, \quad M_{\ell-1} = \begin{bmatrix} \text{---} & \text{---} & \text{---} \\ \text{---} & \text{---} & \text{---} \end{bmatrix}.$$

$U_{\ell-1}$: upsampled map; $\bar{A}_{\ell-1}$: preceding layer's channel average; $M_{\ell-1} = \bar{A}_{\ell-1} \odot U_{\ell-1}$: next mask. $U_{\ell-1}(i, j)$ denotes row i , column j ; indices start at 1.

Why is $U_{\ell-1}(1, 2)$ its computed value? _____

Where $\bar{A}_{\ell-1}$ is zero, what happens to $M_{\ell-1}$? _____

B. Trace the reverse sizes. Each map has one channel per image. Input shape is $(B, 1, x_h, x_w)$; output shape is $(B, 1, y_h, y_w)$. Here B counts images, 1 is the single channel, and x_h, x_w, y_h, y_w are input/output height and width. Fill the empty output sizes; all three steps use $K_\ell = 3$.

Reverse layer	S_ℓ	Input $x_h \times x_w$	Output $y_h \times y_w$	Then multiply by
Conv3 ($\ell = 3$)	1	27×59	29×61	$\bar{A}_2$
Conv2 ($\ell = 2$)	2	29×61		$\bar{A}_1$
Conv1 ($\ell = 1$)	1	59×123		None; normalize

Show one calculation (reverse Conv3 height): _____

Sources. [1] Bernhard Firner, CS 461 Lecture 07 (2026-09-23): “Kernel Size”, “Image Planes”, “Padding”, “Receptive Fields”, “Feature Sizes”, and “Wider Params”.

[2] Bojarski et al., *VisualBackProp*, ICRA 2018, §III / Fig. 2, p. 4703.

[3] Kevin P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022; local PDF revision Apr. 18, 2025. §14.2.1.4–6, Eqs. (14.10), (14.12), (14.15), pp. 471–474; §14.4.2, p. 487.